



JJ jjoseph1608@outlook.com

jjoseph1608@outlook.com

[illegible]



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a virtual private cloud that lets you create your own isolated network within AWS, and it is useful because it gives you full control over your network settings such as IP address ranges, subnets, and security rules, meaning you can keep your resources private and secure while deciding exactly what can and cannot communicate with each other.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create an isolated network environment where I could securely deploy and manage my AWS resources, controlling what traffic was allowed in and out of my network.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how much network configuration was involved before even being able to deploy any resources, as I assumed I could just launch something straight away without needing to set up the underlying network infrastructure first.

This project took me...

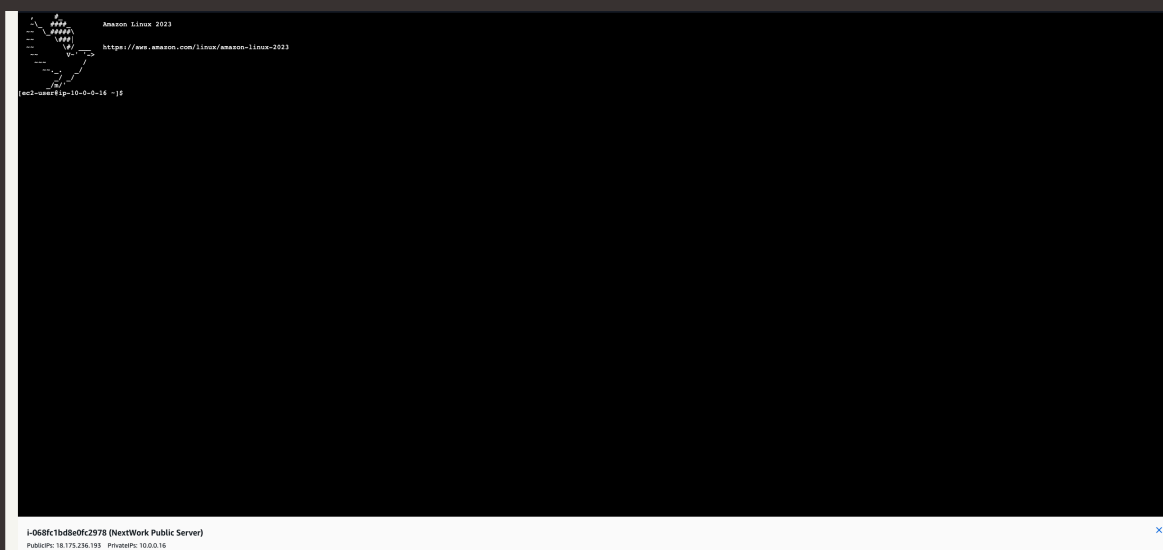
This project took me around an hour to complete, including setting up my VPC, configuring the network settings, and testing the connectivity using curl and ping.



Connecting to an EC2 Instance

Connectivity means the ability of two or more devices or systems to communicate and exchange data with each other. In the context of AWS, it refers to whether your EC2 instances can be reached over the internet or communicate with other instances within the VPC, which depends on how your networking resources like route tables, security groups and internet gateways have been configured.

My first connectivity test was whether I could connect to my public EC2 instance, which is the instance sitting inside my public subnet that should be accessible over the internet through the internet gateway I configured as part of my VPC setup.





EC2 Instance Connect

I connected to my EC2 instance using EC2 Instance Connect, which is a browser based tool built into the AWS console that allows you to connect directly to your EC2 instance and control it through a terminal without needing to use any external software or manage a private key file on your own computer. It makes accessing your instance much quicker and simpler, especially for beginners.

My first attempt at getting direct access to my public server resulted in an error, because my security group did not have a rule allowing inbound SSH traffic on port 22, which is the port that EC2 Instance Connect uses to establish a connection. Without this rule in place, the security group was blocking the connection attempt before it could reach the instance.

I fixed this error by going into my security group's inbound rules and adding a new rule that allowed SSH traffic on port 22, which gave EC2 Instance Connect the permission it needed to establish a connection to my public EC2 instance. Once the rule was saved and the connection was reattempted, it was successful.





Connectivity Between Servers

Ping is a simple command that sends a small signal from one device to another and waits for a response, which tells you whether the two devices can communicate with each other and how long it takes for the signal to travel between them. I used ping to test the connectivity between my public EC2 instance and my private EC2 instance, to check whether the two servers could reach each other within the VPC.

The ping command I ran was ``ping 10.0.1.213``, which sent a signal from my public EC2 instance to my private EC2 instance at that private IP address, to test whether the two instances could communicate with each other within the VPC.

The first ping returned no response, with the terminal just sitting there waiting without receiving any replies back. This meant that my public EC2 instance was unable to communicate with my private EC2 instance, indicating that something in the network configuration was blocking the traffic between the two instances.



jjoseph1608@outlook....

NextWork Student

nextwork.org





Troubleshooting Connectivity

I troubleshooted this by checking my private server's security group inbound rules, where I found that there was no rule allowing ICMP traffic, which is the protocol that ping uses to send and receive signals. I added a new inbound rule to the private security group to allow ICMP traffic from the public server's security group, which then allowed the ping signal to reach the private instance and return a successful response.

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

root@centos4p-10-0-0-16 ~# ping 10.0.1.213
PING 10.0.1.213 (10.0.1.213) 56(84) bytes of data:
64 bytes from 10.0.1.213: icmp_seq=1185 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1186 ttl=127 time=0.228 ms
64 bytes from 10.0.1.213: icmp_seq=1187 ttl=127 time=0.267 ms
64 bytes from 10.0.1.213: icmp_seq=1188 ttl=127 time=0.217 ms
64 bytes from 10.0.1.213: icmp_seq=1189 ttl=127 time=0.211 ms
64 bytes from 10.0.1.213: icmp_seq=1190 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1191 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1192 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1193 ttl=127 time=0.276 ms
64 bytes from 10.0.1.213: icmp_seq=1194 ttl=127 time=0.249 ms
64 bytes from 10.0.1.213: icmp_seq=1195 ttl=127 time=0.221 ms
64 bytes from 10.0.1.213: icmp_seq=1196 ttl=127 time=0.239 ms
64 bytes from 10.0.1.213: icmp_seq=1197 ttl=127 time=0.239 ms
64 bytes from 10.0.1.213: icmp_seq=1198 ttl=127 time=0.247 ms
64 bytes from 10.0.1.213: icmp_seq=1199 ttl=127 time=0.254 ms
64 bytes from 10.0.1.213: icmp_seq=1200 ttl=127 time=0.290 ms
64 bytes from 10.0.1.213: icmp_seq=1201 ttl=127 time=0.250 ms
64 bytes from 10.0.1.213: icmp_seq=1202 ttl=127 time=0.228 ms
64 bytes from 10.0.1.213: icmp_seq=1203 ttl=127 time=0.210 ms
64 bytes from 10.0.1.213: icmp_seq=1204 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1205 ttl=127 time=0.241 ms
64 bytes from 10.0.1.213: icmp_seq=1206 ttl=127 time=0.230 ms
64 bytes from 10.0.1.213: icmp_seq=1207 ttl=127 time=0.220 ms
64 bytes from 10.0.1.213: icmp_seq=1208 ttl=127 time=0.250 ms
64 bytes from 10.0.1.213: icmp_seq=1209 ttl=127 time=0.211 ms
64 bytes from 10.0.1.213: icmp_seq=1210 ttl=127 time=0.212 ms
64 bytes from 10.0.1.213: icmp_seq=1211 ttl=127 time=0.272 ms
64 bytes from 10.0.1.213: icmp_seq=1212 ttl=127 time=0.261 ms
64 bytes from 10.0.1.213: icmp_seq=1213 ttl=127 time=0.300 ms
64 bytes from 10.0.1.213: icmp_seq=1214 ttl=127 time=0.250 ms
64 bytes from 10.0.1.213: icmp_seq=1215 ttl=127 time=0.266 ms
64 bytes from 10.0.1.213: icmp_seq=1216 ttl=127 time=0.264 ms
64 bytes from 10.0.1.213: icmp_seq=1217 ttl=127 time=0.251 ms
64 bytes from 10.0.1.213: icmp_seq=1218 ttl=127 time=0.210 ms
64 bytes from 10.0.1.213: icmp_seq=1219 ttl=127 time=0.220 ms
64 bytes from 10.0.1.213: icmp_seq=1220 ttl=127 time=0.226 ms
64 bytes from 10.0.1.213: icmp_seq=1221 ttl=127 time=0.240 ms
64 bytes from 10.0.1.213: icmp_seq=1222 ttl=127 time=0.244 ms
64 bytes from 10.0.1.213: icmp_seq=1223 ttl=127 time=0.244 ms
64 bytes from 10.0.1.213: icmp_seq=1224 ttl=127 time=0.244 ms
64 bytes from 10.0.1.213: icmp_seq=1225 ttl=127 time=0.244 ms
```

i-068fc1bd8e0fc2978 (NextWork Public Server)
PublicIP: 18.175.236.193 PrivateIP: 10.0.0.16



Connectivity to the Internet

Curl is a command line tool used to send and receive data over the internet. Think of it like a browser, but in the terminal instead of a window.

I used curl to test the connectivity between my local machine and the AWS service, as it allows me to send requests and verify that I'm getting the expected response back.

Ping vs Curl

Ping and curl are different because ping simply checks whether a server is reachable and measures the response time, whereas curl actually sends a full request and retrieves data, so curl can test whether a web service or API is working correctly, not just whether the server is online.



Connectivity to the Internet

I ran the curl command `curl learn.nextwork.org` which returned the HTML content of the webpage, confirming that the connection was successful and the server was responding as expected.

```
font-size: 3.75rem;
font-weight: 600;
line-height: 1;
margin: 1.5rem 0;
color: #111827;
}

.no-js-description {
  font-size: 1.25rem;
  color: #666666;
  line-height: 1.4;
  margin: 0;
  width: 80%;
}

.no-js-shield {
  width: 2.4rem;
  height: 2.4rem;
}

.no-js-logo-image {
  width: 2rem;
  height: 2rem;
}

@media (max-width: 768px) {
  .no-js-heading {
    font-size: 2.5rem;
  }

  .no-js-description {
    font-size: 1.125rem;
    width: 100%;
  }

  .no-js-container {
    padding: 0 1rem;
  }
}

</script>
<div class="no-js-overlay">
  <div class="no-js-logo">
    
  </div>
  <div class="no-js-container">
    <div
      class="no-js-shield no-js"
      alt="Shield with gap icon"
      class="no-js-shield"
    />
    <p class="no-js-heading">JavaScript is required, whaaaaaat!</p>
    <p class="no-js-description">
      NextWork requires JavaScript to function properly. Please enable
      JavaScript or disable any script-blocking tools like Brave Shields
      to use this site.
    </p>
  </div>
</div>
</script>
</html>
curl -userip 10-0-0-16 -H
```

i-068fc1bd8dfc2978 (NextWork Public Server)
PublicIP: 18.175.238.193 PrivateIP: 10.0.0.16



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

